BrainAI

ProSuite Industrial

PROSUITE INDUSTRIAL

PRODUCT DOCUMENTATION

BrainAI (Inhaber: Sascha Alexander Köhne)

BrainAI Inhaber: Sascha Alexander Köhne	PROSUITE INDUSTRIAL	🟢 PUBLIC CLASSIFICATION
Core Architecture & Development	**Technical Documentation**	Access Level: Public / Beta

---

Revision History

Version	Date	Author	Status	Description
1.0.0	2026-02-28	S. Köhne	Stable	Initial Release: ProWB System & Integration Guide

1. Product Overview: Pro Web Builder (ProWB)

The **ProWB** is a static site generator and monolith builder written entirely in pure C. It was explicitly designed to encapsulate our cryptographic WebAssembly modules (ProED, ProChat, ProKey, ProHash) within a highly secure, isolated, and Zero-Trust compliant web environment.

Unlike conventional web frameworks (such as React or Angular), which are inherently vulnerable to supply-chain attacks via external dependencies, ProWB generates a single, self-contained, static `index.html`. All CSS styles, JavaScript routines, and HTML views are injected inline into this monolith during the build process, eliminating external attack vectors.

2. Directory Structure (Source Tree)

The ProWB enforces a strict folder layout within the source directory (`src_dir`). Any deviation from this structure is deliberately ignored by the builder to prevent unauthorized file injections.

```
prowb/src/
├── parts/      # HTML skeletons (Header, Footer, Navigation)
├── css/        # Cascading Style Sheets (injected inline)
├── js/         # JavaScript logic & WASM bridges (injected inline)
├── views/      # Native HTML views (Dashboards, UI-Panels)
└── docs/       # Markdown manuals (parsed natively to HTML)
```

2.1 The "00_ to 99_" Sorting System

Files located in `views/`, `docs/`, `css/`, and `js/` are strictly read in alphabetical order. To control the load order (crucial for CSS/JS cascading) and the rendering sequence of navigation buttons, ProWB utilizes a numerical prefix system:

- `00_vars.css` is loaded before `99_theme.css`.
- `00_view_home.html` appears at the very top of the generated navigation.

Security Feature: The C parser intelligently strips this numerical prefix during compilation. For example, `00_view_home.html` is securely sanitized into the clean JavaScript ID `view_home` for internal routing.

3. Structure of HTML Files (`/views` & `/parts`)

3.1 Framework Components (`/parts`)

These files form the skeleton of the monolith:

- **`header.html`**: Contains the `<head>` section, meta tags, and the opening of the `<body>`.
- **`footer.html`**: Closes the document safely.
- **`nav.html`**: Contains the template for the navigation bar. The ProWB searches for two specific placeholders here:
- `{{NAV_VIEWS}}`: The C engine automatically injects the buttons for all files from the `/views` directory here.
- `{{NAV_DOCS}}`: The C engine automatically injects the buttons for all Markdown files from the `/docs` directory here.

3.2 UI Views (`/views`)

Since the builder concatenates all HTML files from `/views` sequentially into the `index.html`, these files must define themselves as toggleable blocks.

Mandatory Structure for a View File (e.g., `10_view_prokey.html`):

```
<section id="view_prokey" class="view-content" style="display:none;">
    <h2>Hardware Entropy (ProKey)</h2>
    </section>
```

- **Critical:** The `id` must exactly match the sanitized filename (excluding the `00_` prefix and the `.html` extension). The `app.js` routing logic relies on this ID to make the block visible when the corresponding navigation button is clicked.

4. Preparing Markdown Documents (`/docs`)

Documents inside the `/docs` directory (e.g., `90_License.md`) are processed by the military-hardened `md_parser.c` engine.

4.1 Automatic HTML Wrapping

Unlike the HTML views, you do **not** need to manually add `<section>` tags to Markdown files. The parser automatically generates the following secure wrapper around your parsed document:

```
<section id="doc_License" class="view-content" style="display:none;">
    <div class="markdown-body">
        </div>
</section>
```

4.2 Best Practices for ProWB Markdown (GFM)

- **Headings:** A space must strictly follow the hash symbol (`# H1`, `## H2`).
- **Safe Filenames:** Avoid special characters or spaces in `.md` filenames (e.g., use `05_Compliance_Guide.md`). While the parser sanitizes dangerous characters into underscores (_), clean names prevent routing anomalies.
- **Code Blocks:** Always specify the language (e.g., ```c). The parser translates this into `, which is required for frontend syntax highlighters (like PrismJS).

5. The Build Process

The ProWB is executed via the command line interface (CLI) and requires three mandatory parameters:

1. **Source Directory:** The path to the src folder.
2. **Output Directory:** The base path where the compiled project will be stored.
3. **Core Name:** The identifier of the WASM core to be loaded (e.g., proedc).

Execution Example (Linux/Windows):

```
./prowb ./src/prowb ./dist proedc
```

- **Injection Mechanism:** The builder automatically writes the global constant `<script>const PRO_WASM_CORE = 'proedc';</script>` into the

output file. This ensures the JavaScript routing logic knows exactly which cryptographic module to mount upon initialization.

© 2026 Sascha Köhne	✉️ **Contact: koehne83@googlemail.com**	🛡️ **ProTU-Verified**
System Architect	BrainAI Inhaber : Sascha Alexander Köhne	We don't need **BRUTEFORCE**, we know **PHYSICS**